



Yapay Sinir Ağları ile Akıllı Tür Tanıma

Fotoğrafı yüklenen evcil hayvanın türünü saniyeler içinde tespit etmek ve kullanıcıya o türe dair ömür, kilo, köken gibi detaylı bilgileri sunan bir rehber oluşturmak.



Projenin Amacı

Kullanıcılar evcil hayvanlarının fotoğrafını sisteme yüklüyor. Model, yüklenen görseli analiz ederek türü saniyeler içinde tespit ediyor ve o türe özel kapsamlı bir bilgi kartı sunuyor.

⚡ Anında Tür Tespiti

Yüklenen fotoğraf model tarafından işlenir ve 37 farklı kedi/köpek cinsi arasından en yüksek güven oranıyla eşleşme bulunur. İşlem sunucu olmadan, doğrudan kullanıcının cihazında gerçekleşir.

📄 Detaylı Tür Rehberi

Tespit edilen tür için sistem; ortalama ömür, kilo aralığı, coğrafi köken, karakteristik görsel özellikler ve bakım ipuçları gibi zengin bilgileri tek bir kart üzerinde derleyerek sunar.

🎯 Kullanıcı Odaklı Tasarım

Teknik altyapı ne kadar karmaşık olursa olsun kullanıcı deneyimi sade tutuldu: bir görsel yükle, sonucu gör. Veteriner klinikleri, evcil hayvan sahipleri ve meraklı herkes için uygundur.

Teknolojik Altyapı

Projenin en kritik tasarım kararlarından biri şuydu: **sunucu olmadan çalışmak**. Tüm yapay zeka işlemleri kullanıcının tarayıcısında gerçekleşiyor. Bu sayede backend maliyeti, gecikme süresi ve güvenlik riski sıfıra yaklaşıyor.



HTML & CSS

Kullanıcı arayüzü saf HTML5 yapısı ve modern CSS3 özellikleriyle oluşturuldu. Responsive tasarım sayesinde masaüstü ve mobil cihazlarda tutarlı bir deneyim sunuluyor.



JavaScript (TensorFlow.js)

Google tarafından geliştirilen TensorFlow.js kütüphanesi, eğitilmiş Python modelini tarayıcıda çalıştırmamızı sağladı. Model ağırlıkları `model.json` formatında yükleniyor ve tahmin işlemi milisaniyeler içinde tamamlanıyor.



Sunucusuz Mimari

Geleneksel yapay zeka uygulamalarının aksine görüntü hiçbir sunucuya gönderilmiyor. İstemci tabanlı bu yaklaşım; kullanıcı gizliliğini koruyor, internet bağlantısı kesimleri karşısında dirençli kılıyor ve bulut maliyetini ortadan kaldırıyor.

Veri Seti: Oxford-IIIT Pet Dataset

Dataset'e Genel Bakış

Oxford Üniversitesi ve IIIT Hyderabad tarafından hazırlanan bu benchmark veri seti, bilgisayarlı görü alanında dünya genelinde yaygın biçimde kullanılmaktadır.

- 37 farklı kedi ve köpek cinsi
- Her sınıf için yaklaşık 200 etiketli görsel
- Toplam ~7.400 yüksek kaliteli fotoğraf
- Piksel düzeyinde segmentasyon maskeleri
- Sınır kutusu (bounding box) etiketleri

Google Colab Süreçleri

Model, Python ekosistemi kullanılarak Google Colab ortamında inşa edildi. Colab'ın ücretsiz GPU desteği, eğitim süresini yerel bilgisayara kıyasla onlarca kat kısalttı.

1

1. Veri Hazırlama

Oxford-IIIT dataset indirildi, görseller 224×224 boyutuna yeniden ölçeklendirildi ve normalize edildi. Eğitim/doğrulama split'i uygulandı.

2

2. Model İnşası

MobileNetV2 base modeli ImageNet ağırlıklarıyla yüklendi.

3

3. Eğitim & Doğrulama

Model belirlenen hiperparametrelerle eğitildi.

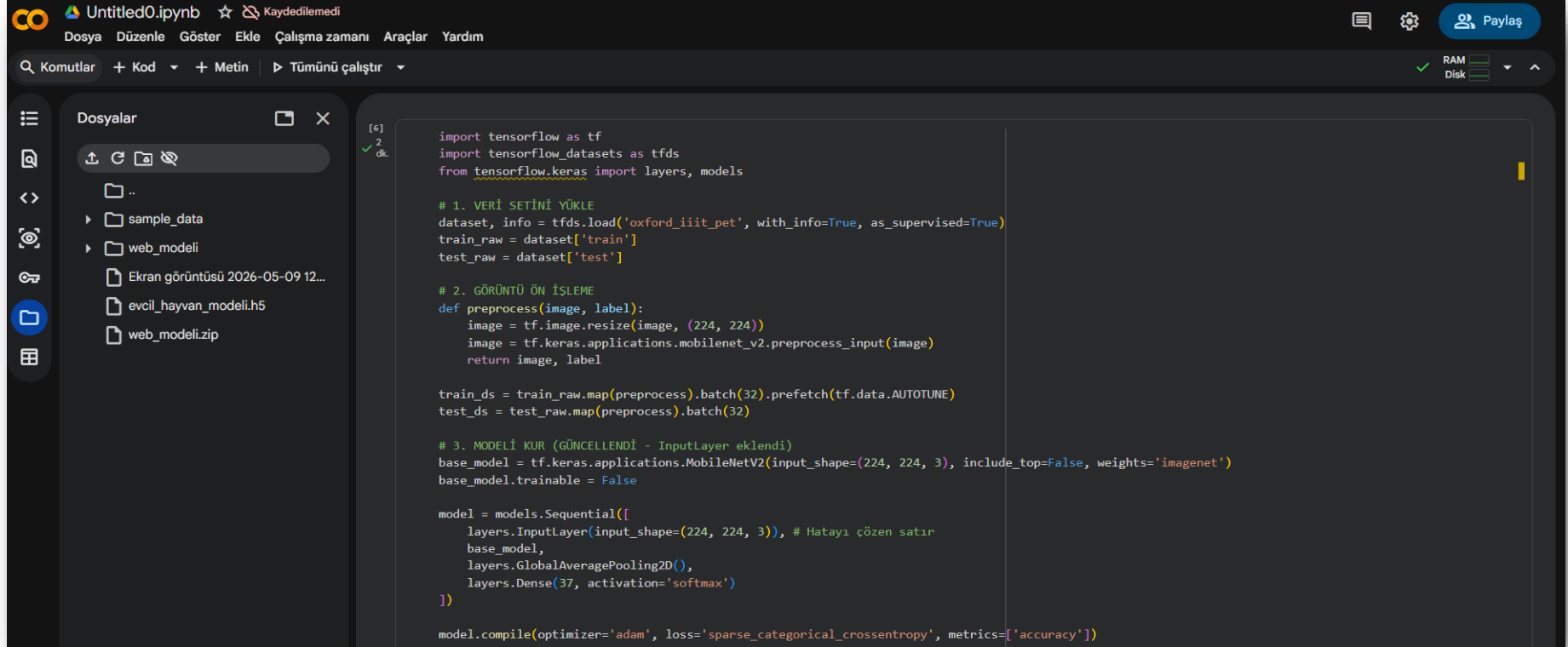
4

4. Web'e Dönüştürme

Eğitilen model, `tensorflowjs_converter` aracıyla `model.json` + binary weights formatına dönüştürülerek tarayıcıda kullanıma hazır hale getirildi.



Bu kısımda, görselleri tanıma konusunda kendini kanıtlamış hazır bir model yapısını temel alarak projemizi kurguladık. Resimleri modelin anlayabileceği standart boyuta getirip, sistemin mevcut zekasını 37 farklı hayvan türünü tanıyacak şekilde özelleştirdik. Amacımız, modeli sıfırdan eğitmek yerine var olan tecrübesini kullanarak hızlı ve doğru sonuçlar üretmesini sağlamaktır.



The screenshot displays a Jupyter Notebook environment with a dark theme. The top bar shows the notebook title 'Untitled0.ipynb' and a 'Kaydedilemedi' (Not Saved) status. The left sidebar contains a file explorer with a tree view showing folders like 'sample_data' and 'web_modeli', and files like 'Ekran görüntüsü 2026-05-09 12...' and 'web_modeli.zip'. The main area shows a Python script with the following code:

```
[6]
2
✓ dk.

import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models

# 1. VERİ SETİNİ YÜKLE
dataset, info = tfds.load('oxford_iiit_pet', with_info=True, as_supervised=True)
train_raw = dataset['train']
test_raw = dataset['test']

# 2. GÖRÜNTÜ ÖN İŞLEME
def preprocess(image, label):
    image = tf.image.resize(image, (224, 224))
    image = tf.keras.applications.mobilenet_v2.preprocess_input(image)
    return image, label

train_ds = train_raw.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
test_ds = test_raw.map(preprocess).batch(32)

# 3. MODELİ KUR (GÜNCELLENDİ - InputLayer eklendi)
base_model = tf.keras.applications.MobileNetV2(input_shape=(224, 224, 3), include_top=False, weights='imagenet')
base_model.trainable = False

model = models.Sequential([
    layers.InputLayer(input_shape=(224, 224, 3)), # Hatayı çözen satır
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(37, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Hazırladığımız modeli 5 turda eğittik ve her adımda resimleri küçük gruplar halinde işleyerek süreci verimli hale getirdik.

```
[6] 2 dk.
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# 4. EĞİTİMİ BAŞLAT
print("Eğitim başlıyor... GPU devrede.")
model.fit(train_ds, epochs=5, validation_data=test_ds)

# 5. MODELİ KAYDET
model.save('evcil_hayvan_modeli.h5')
print("Model 'evcil_hayvan_modeli.h5' adıyla kaydedildi!")

... Eğitim başlıyor... GPU devrede.
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/input_layer.py:27: UserWarning: Argument `input_shape` is deprecated. Use `shape` instead.
  warnings.warn(
115/115 ————— 37s 249ms/step - accuracy: 0.7310 - loss: 1.1028 - val_accuracy: 0.8700 - val_loss: 0.4801
Epoch 2/5
115/115 ————— 20s 170ms/step - accuracy: 0.9296 - loss: 0.2830 - val_accuracy: 0.8825 - val_loss: 0.3886
Epoch 3/5
115/115 ————— 22s 192ms/step - accuracy: 0.9587 - loss: 0.1797 - val_accuracy: 0.8904 - val_loss: 0.3581
Epoch 4/5
115/115 ————— 39s 178ms/step - accuracy: 0.9791 - loss: 0.1251 - val_accuracy: 0.8932 - val_loss: 0.3443
Epoch 5/5
115/115 ————— 20s 176ms/step - accuracy: 0.9880 - loss: 0.0917 - val_accuracy: 0.8940 - val_loss: 0.3362
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend you to use the Keras 2.x format instead.
Model 'evcil_hayvan_modeli.h5' adıyla kaydedildi!

[7] 6 sn.
!tensorflowjs_converter --input_format=keras evcil_hayvan_modeli.h5 web_modeli
!zip -r web_modeli.zip web_modeli

11:48:19.964713: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin
```

Transfer Learning: MobileNetV2

Neden Transfer Learning?

Derin öğrenme modellerini sıfırdan eğitmek için milyonlarca örnek ve günlerce hesaplama süresi gerekir. Transfer Learning bu engeli aşmanın en etkili yoludur: başka bir görev için eğitilmiş bir modelin öğrendiği bilgiyi yeni bir göreve aktarırsınız.

MobileNetV2, Google tarafından mobil ve gömülü sistemler için optimize edilmiş, hafif ama güçlü bir mimariye sahiptir.



MobileNetV2 Mimarisi

Giriş: 224×224×3

Parametre Sayısı: ~3.4M

ImageNet Top-5 Doğruluğu: %90.1

Çıkış Katmanı: 37 sınıf (Softmax)

Fine-Tuning Avantajları

- Sıfırdan eğitime göre **10x daha az** veri ihtiyacı
- Eğitim süresi saatler yerine **dakikalar**
- Daha yüksek başlangıç doğruluğu
- Küçük veri setlerinde overfitting riski azalır
- Colab ücretsiz GPU ile eğitilebilir

Hiperparametreler: Eğitim Ayarları

Bu değerler deneyim, teori ve deneme-yanılma süreciyle belirlendi.

5

Epoch

Modelin tüm eğitim setini kaç kez baştan sona gördüğünü belirtir. 5 epoch, transfer learning senaryosunda overfitting yaşamadan yeterli öğrenmeyi sağladı. Daha fazla epoch validation loss'u artırıyordu.

32

Batch Size

Her ağırlık güncellemesinde aynı anda işlenen görsel sayısıdır. 32'lik batch size, bellek kullanımı ile gradient hesaplama kararlılığı arasında iyi bir denge kurdu. Colab GPU belleğiyle uyumlu optimum değerdir.

224

Girdi Boyutu (px)

Her görsel 224×224×3 boyutuna yeniden ölçekleniyor. Bu boyut MobileNetV2'nin standart giriş formatıdır. Daha büyük boyut daha yüksek hesaplama maliyeti demektir; daha küçük boyut ise önemli detayların kaybolmasına neden olur.

Öğrenme Hızı (Learning Rate)

İlk aşamada base model dondurulduğundan görece yüksek bir learning rate kullanıldı. Fine-tuning aşamasında ise çok daha düşük bir değere ($1e-5$) inildi; aksi hâlde önceki öğrenmeler silinir.

Optimizer & Loss Fonksiyonu

Adam optimizer, adaptif öğrenme hızı sayesinde her parametre için ayrı güncelleme yaparak hızlı ve kararlı bir yakınsama sağladı. Çok sınıflı sınıflandırma için `categorical_crossentropy` loss fonksiyonu kullanıldı.

$$\text{Loss} = - \sum_{i=1}^{37} y_i \log(\hat{y}_i)$$

Uygulama Örneđi: Persian Cat



CANLI SİSTEM ÇIKTISI

Persian Cat — %98.1 Güven

Sistem, yüklenen görseli işledikten sonra aşağıdaki zengin bilgi kartını milisaniyeler içinde oluşturdu:

Tür Adı	Persian Cat (İran Kedisi)
Güven Oranı	%98.1
Ortalama Ömür	12 – 17 yıl
Kilo Aralığı	3 – 6 kg
Köken	İran (Eski Persia)
Tüy Yapısı	Uzun, kalın ve lüks
Yüz Yapısı	Basık (brachycephalic)
Karakter	Sakin, uysal, sevecen



%98.1 güven oranı, modelin bu tür için son derece belirleyici görsel özellikler (basık yüz, uzun tüy) yakaladığını gösteriyor.